AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

**Amrita School of AI**
COIMBATORE CAMPUS

23AID304 · HIGH-PERFORMANCE AND CLOUD COMPUTING

# Getting Started

Everything you need in week one. Where the course material lives, how to get it, how to work on the coding exercises, and how to submit them.

Prepared by **Dr. Abhijith Anandakrishnan**
Assistant Professor, Amrita School of AI

COURSE
23AID304 High-Performance and Cloud Computing

REPOSITORY
github.com/Amrita-School-of-AI/hpc-23aid304

READ THIS
In week one, before the first lab

TIME NEEDED
About 45 minutes, once

OFFERING
Odd Semester 2026-27

# Read this once, in week one

Everything for this course lives in one public GitHub repository. This guide takes you from a laptop with nothing set up to a working copy of the material, a compiler that runs the tests, and a submission you can be confident in.

Work through it in order. It takes about forty-five minutes and you never have to do it again.

**BY THE END OF THIS GUIDE YOU WILL HAVE**

- A GitHub account with an SSH key that works.
- The whole course repository on your machine.
- A compiler and MPI installed, and the public tests running.
- The `gh student` tool logged in, so that accepting and submitting an item is one command each.
- Submitted the warm-up assignment and read the grader's report on it.

# Part 1: What is in the repository

`github.com/Amrita-School-of-AI/hpc-23aid304`

It is public. You do not need permission and you do not need to be added to anything.

```
hpc-23aid304/
├── handbook/          the course handbook: week plan, marks, policies, setup
├── notes/             five notes booklets, Units 0 to 4
├── slides/            the lecture decks
├── labs/              lab sheets, and the cluster setup guide
├── assignments/       the assignment briefs
├── mini-project/      the course project catalogue
└── exercises/         the coding exercise statements, for reading
```

You do not work in this repository. Each lab, exercise and assignment is its own repository, created for you by Classroom 50 when you accept it (Part 3.2).

| FOLDER | WHAT IT IS | WHEN YOU NEED IT |
|---|---|---|
| `handbook/` | **Start here.** Week-by-week plan, how marks work, policies, what to install | Week 1 |
| `notes/` | Self-contained booklets. You are **not** expected to buy or read a textbook | Throughout |
| `slides/` | What was on screen in class | After each lecture |
| `labs/` | The lab sheet for each session, with what to submit | Before each lab |
| `assignments/` | Four assignments, part of the 10 percent lab and assignment component | Weeks 3, 6, 9, 12 |
| `exercises/` | Auto-graded coding exercises | Throughout |
| `mini-project/` | Twelve project options, **30 percent of the course** | From week 6 |

**NOTE · THE NOTES ARE THE TEXTBOOK**

Units 0 to 4 are written to be read on their own. Unit 0 exists because the rest of the course assumes things that are easy to forget: cache lines, threads, and how to time a program honestly. Read it in week one even if it looks familiar.

## Inside one exercise

```
exercises/ex06-openmp-reduction/
├── ex06-openmp-reduction.pdf   the problem statement. READ THIS FIRST
├── starter/reduction.c          the file you edit
├── include/reduction.h          the interface. Do not change it
├── tests/public.c               the tests you can run yourself
└── Makefile                     `make check` compiles and runs those tests
```

# Part 2: A GitHub account and an SSH key

You can download the material without any of this. You need it for Classroom 50, which creates your assignment repositories under your GitHub account.

## 2.1 Create the account

Go to **github.com** and sign up. Use a username you are willing to show an employer: it becomes part of your professional record. Add your Amrita email so you can claim the free **GitHub Student Developer Pack** later.

## 2.2 Make an SSH key

An SSH key is a pair of files. The **private** key stays on your laptop forever. The **public** key you give to GitHub, which then recognises your machine without a password.

**Linux, macOS, or Windows with WSL:**

```
ssh-keygen -t ed25519 -C "your.email@example.com"
```

Press Enter three times to accept the default location. You may set a passphrase; if you do, you will be asked for it when you push.

**Windows without WSL:** open PowerShell and run the same command.

> **PITFALL · NEVER SHARE THE PRIVATE KEY**
>
> `~/.ssh/id_ed25519` is private. `~/.ssh/id_ed25519.pub` ends in `.pub` and is the one you share. If you ever paste a key into a chat, a website or a repository, delete it from GitHub and generate a new one. The same discipline applies to cloud credentials later in this course.

## 2.3 Give the public key to GitHub

Print it:

```
cat ~/.ssh/id_ed25519.pub
```

You get one line starting `ssh-ed25519 AAAA...`. Copy the whole line.

On GitHub: click your avatar, top right, then

**Settings → SSH and GPG keys → New SSH key**

Give it a title such as `laptop`, leave the type as *Authentication Key*, paste the line into the box, and click **Add SSH key**.

## 2.4 Check it works

```
ssh -T git@github.com
```

The first time it asks about authenticity: type `yes`. You should then see:

```
Hi <your-username>! You've successfully authenticated, but GitHub does not
provide shell access.
```

That message is a success, including the part about shell access.

**PITFALL · IF IT SAYS PERMISSION DENIED (PUBLICKEY)**

Three usual causes, in order of likelihood: you pasted the private key instead of the `.pub` one; you copied only part of the line; or the key is not being offered by the agent. For the last case:

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

## 2.5 Tell git who you are

```
git config --global user.name  "Your Name"
git config --global user.email "your.email@example.com"
```

# Part 3: Get the material

## 3.1 Just reading it

If you only want the notes and slides, download the ZIP from the green **Code** button on the repository page. No account needed.

## 3.2 Where your work lives: Classroom 50

The course repository is read-only for you. Your work lives in **Classroom 50**, the School's replacement for GitHub Classroom: every lab, exercise and assignment is a private repository created for you the moment you accept it, under the `Amrita-School-of-AI` organisation. Accepting, submitting and reading your grade all happen through one small tool:

```
gh extension install foundation50/gh-student
gh student login          # opens the browser once; grants the scopes Classroom 50 needs
gh student whoami         # should print your GitHub username
```

You will receive an invitation to the organisation by email at your college address. Accept it before the first lab. Your GitHub username must match the one on the roster; if you use a different account the repositories will not be created for you.

## 3.3 Getting updates during the semester

New material appears in the course repository; `git pull` in your clone of it is all that is needed for reading. Each assignment repository is created from its template at the moment you accept, so accept an item only when you are about to start it.

# Part 4: Install what you need

Everything is free.

**Ubuntu, Debian, Linux Mint, or WSL:**

```
sudo apt update
sudo apt install build-essential gdb git openmpi-bin libopenmpi-dev python3 python3-yaml
```

**macOS:**

```
xcode-select --install
brew install libomp open-mpi
```

**Windows:** install WSL2 with Ubuntu (`wsl --install` in an administrator PowerShell), then follow the Ubuntu instructions inside it. Do not fight native Windows for this course.

Check:

```
gcc --version
mpicc --version
echo | gcc -fopenmp -x c - -o /dev/null && echo "OpenMP works"
```

> **NOTE · CUDA IS NOT ON THIS LIST**
>
> The GPU exercises run on the School's cluster, not your laptop. Lab 2 shows you how to log in and how to ask the scheduler for a GPU. You do not need an NVIDIA card of your own.

# Part 5: Working on an item

## 5.1 Accept, then clone

```
gh student accept Amrita-School-of-AI aim-a-hpc ex06-openmp-reduction
```

Use `aim-b-hpc` if you are in the B section. The command creates `Amrita-School-of-AI/aim-a-hpc-ex06-openmp-reduction-<your-username>` and prints the clone URL. Clone it with `gh repo clone <that name>` or with the `git@` address; the plain `https://` clone asks for a password and fails.

## 5.2 Read the statement

Every repository has a `README.md` that is the full statement: the problem, the traps, and a table of exactly what is tested and for how many marks. Read it before the code. The last section of every README says which parts are graded and which are for you to do on the cluster and think about.

## 5.3 Edit only what you are told to edit

The README names the files that are yours, usually `src/*.c`, sometimes `answers.yaml` or a CSV under `results/`. `.gitignore` and `.github/` belong to the course and are put back the way they were every time you submit; changing them achieves nothing. Test files under `tests/` are copies of the public tests; the grader uses its own copy, so editing them cannot change your mark.

## 5.4 Run the public tests

```
make check
```

That compiles your file with the same flags the grader uses, warnings as errors, and runs the public tests. The grader also runs hidden tests on inputs you have not seen, so a clean `make check` is necessary but not sufficient.

# Part 6: Submitting

```
git add -A
git commit -m "say what you changed"
gh student submit
```

`gh student submit` pushes your commit and tags it. About a minute later:

- a **green tick or red cross** appears on the commit on GitHub;
- a **Release** appears on your repository, named after the submission tag, whose text is the grader's report: one line per check, PASS or FAIL, the marks for it, and for every failure the reason, such as which test cases failed or what the measured speedup was against what was required;
- the same report is in the **Actions** tab under the run's summary.

Read the report, fix the file it names, commit, submit again. You may submit as many times as you like before the deadline; the last submission counts. Before a second submission run `git pull --rebase origin main` first, because the submission itself added a commit.

## What the grader checks

| KIND OF CHECK | WHAT IT DOES |
|---|---|
| Compiles cleanly | `-Wall -Wextra -Werror`; a failure here stops everything after it |
| Required constructs | the mechanism the item teaches is actually in the source, comments not counted |
| Public tests | the ones `make check` ran |
| Hidden tests | the same behaviour on inputs you have not seen: odd sizes, tiny sizes, awkward rank counts |
| Determinism | repeated runs at several thread counts must agree |
| Speedup | the faster version really is faster, by a modest factor, on a two-core machine |
| Analysis | your program computes speedup, efficiency, Karp-Flatt or the roofline exactly on hidden data |
| Committed results | the CSV you produced on the cluster has the right shape |
| Answers | numbers in `answers.yaml` against a key, with tolerance |

Speedup checks run only after the correctness checks for the same code have passed; a fast wrong answer scores nothing. Marks that need a GPU are graded on the cluster after the deadline, and the report says so.

Some items are recorded in the gradebook as points rather than marks when they have half-mark checks (a 10-mark item as 20 points). The report always shows both.

# Part 7b: How the marks work

| COMPONENT | WEIGHT | WHAT IT IS |
|---|---|---|
| Continuous evaluation tests | **30%** | Three tests on **Evolveus**, around weeks 5, 10 and 14 |
| Lab and class assignments | **10%** | Assignments 6%, coding exercises 2%, lab sheets 2% |
| Course project | **30%** | First review 10% around week 8, final review 20% in the last week |
| End-semester examination | **30%** | Three hours, all four units |

There is **no separate mid-term**. The three continuous evaluation tests replace it.

> **PITFALL · THE CODING EXERCISES ARE WORTH 2 PERCENT, AND YOU SHOULD STILL DO ALL OF THEM**
>
> They are the cheapest possible way to find out that you do not understand something, and the continuous evaluation tests ask you to reason about exactly the things the exercises make you build. Every year some students skip the small components and are then surprised by a test on material they never practised.

The full breakdown, including what each project review is marked on, is in `handbook/course-handbook.pdf`.

# Part 8: Rules

**Late work.** Marked out of 80 percent within 48 hours; not accepted after that without documented cause. Tell me before the deadline, not after.

The documented-cause clause was applied to every Classroom 50 item on 2 November 2026. The School's GitHub Actions quota ran out on 22 October, so nothing submitted after 21:53 that evening was graded, and the VPN that reaches `asaicompute` is down. All 23 graded items therefore run to 7 November 2026, the 80 percent factor applies across that whole window instead of 48 hours, and work blocked by the grading outage counts as on time. `classroom-items.pdf` has the weighting table.

**Collaboration.** Discussing ideas, approaches and errors is encouraged and is how people actually learn. Submitting code you did not write is not. Every submission is compared against every other and against the reference, using a measure that ignores renamed variables and reformatting.

**AI tools.** You may use them to learn, to explain an error, or to explore an approach. You must be able to explain every line you submit. If you cannot, the work is not yours regardless of how it was produced.

**Never commit a credential.** Not in this course, not ever. A key pushed to a public repository is scanned and abused within minutes.

# Part 9: When something goes wrong

Bring me, in this order:

1. What you expected to happen.
2. What actually happened.
3. The **exact** error text, copied as text.
4. The smallest program that still shows the problem.

Producing item 4 solves the problem about half the time on its own.

## Things that catch nearly everyone

| SYMPTOM | CAUSE |
|---|---|
| No speedup at all from OpenMP | You forgot `-fopenmp`. Every `#pragma omp` is silently ignored without it |
| `Permission denied (publickey)` | See Part 2.4 |
| Push rejected, `fetch first` | Submitting added a commit; `git pull --rebase origin main`, then push again |
| No Release after submitting | Wait two minutes, then check the Actions tab; a red run means the grader itself failed, tell me |
| Tests pass sometimes, fail other times | A data race. That is a real bug, not bad luck, and the grader will find it |
| `undefined reference to ...` | A linker error. A `.c` file or a library such as `-lm` is missing from the command |

THE FIVE COMMANDS YOU WILL USE MOST

`make check` run the public tests in an item repository · `python3 tools/check.py` check everything at once · `git add . && git commit -m "..." && git push` save and publish your work · `git fetch upstream && git merge upstream/main` pull new course material · `ssh -T git@github.com` confirm your key still works

Welcome to the course.

**Dr. Abhijith Anandakrishnan** Assistant Professor, Amrita School of AI